

DSA & Advanced DSA Roadmap

Phase 16 — Dynamic Programming

Learning Objectives

By the end of this phase, learners will be able to:

- Identify overlapping subproblems and optimal substructure, the two properties that make a problem solvable with dynamic programming.
- Implement a DP solution two ways: top-down with memoization (recursion plus a cache) and bottom-up with tabulation (iterative table-filling), and explain the trade-offs between them.
- Design DP state and transitions for 1D sequences, 2D grids, two-sequence comparisons, and tree-shaped inputs.
- Recognize the 0/1 knapsack pattern and distinguish it from the fractional knapsack problem (Phase 15), explaining precisely why greedy fails here.
- Solve Easy to Medium level DP problems independently, and approach Hard DP problems (bitmask state, tree DP) by identifying the state space before attempting transitions.

Implementation & Internals

Core Concepts

- Memoization — top-down recursion with a cache keyed by function arguments, so each distinct subproblem is computed once and reused; closely related to Phase 9's recursion tree, now with duplicate branches eliminated.
- Tabulation — bottom-up iteration that fills a table of subproblem answers in dependency order, avoiding recursion overhead entirely.
- State Design — deciding what a DP state must capture to make the problem 'forget-safe': two states that agree on all relevant captured information must have the same optimal future, regardless of how they were reached.
- Transition Design — the recurrence relating a state's answer to the answers of smaller/simpler states; this is the actual algorithmic content of a DP solution.

Implementation Exercises

- Take a Phase 0/Phase 9 exponential recursive solution (e.g. naive Fibonacci) and convert it to memoized, then to fully tabulated, comparing time complexity at each stage.
- Implement a generic memoization decorator/wrapper from scratch (without using a language's built-in caching utility) and apply it to at least two different recursive functions.
- For a chosen 2D DP problem, implement both the full $O(n^2)$ table version and a space-optimized $O(n)$ rolling-array version, and verify they produce identical answers.

Pattern Mastery

Pattern 1: 1D DP

Concepts

- A single-index state, where the answer at position i depends on a small, fixed number of earlier positions.

- Recognizing the 'adjacent choices conflict' shape: deciding whether to include position i forces a decision about which earlier positions remain available.

Practice Problems

- Given an array of house values, find the maximum sum that can be robbed without robbing two adjacent houses.

LeetCode

- [House Robber — leetcode.com/problems/house-robber](https://leetcode.com/problems/house-robber)

Pattern 2: 2D DP

Concepts

- A two-index state, typically comparing two sequences position by position, where the transition depends on whether the current characters/elements match.

Practice Problems

- Given two strings, find the length of their longest common subsequence.

LeetCode

- [Longest Common Subsequence — leetcode.com/problems/longest-common-subsequence](https://leetcode.com/problems/longest-common-subsequence)

Pattern 3: Knapsack Pattern

Concepts

- The 0/1 knapsack shape: each item is either fully included or fully excluded, in contrast to Phase 15's fractional knapsack where items could be split.
- Directly resolving Phase 15's open counterexample: proving why greedy (sort by ratio, take greedily) fails here, and why DP's 'try both choices, take the best' is required instead.

Practice Problems

- Given an array of positive integers, determine whether it can be partitioned into two subsets with equal sum (equivalent to 0/1 knapsack with target = total sum / 2).

LeetCode

- [Partition Equal Subset Sum — leetcode.com/problems/partition-equal-subset-sum](https://leetcode.com/problems/partition-equal-subset-sum)

Pattern 4: LIS Pattern

Concepts

- Finding the longest subsequence satisfying an ordering constraint (here, strictly increasing), where the $O(n^2)$ DP solution tracks, for each position, the best subsequence ending there.
- Recognizing the $O(n \log n)$ optimization using binary search (Phase 7) over a list of subsequence tails, as a preview of how DP and other techniques often combine.

Practice Problems

- Given an integer array, return the length of the longest strictly increasing subsequence.

LeetCode

- [Longest Increasing Subsequence — leetcode.com/problems/longest-increasing-subsequence](https://leetcode.com/problems/longest-increasing-subsequence)

Pattern 5: Grid DP

Concepts

- A 2D state indexed by grid position, where each cell's answer depends on a small number of neighboring cells (typically up/left, or the row below).
- Recognizing the same recurrence shape across superficially different problems: counting paths, minimizing path cost, and triangle-shaped grids all reduce to 'combine this cell with the best of its valid neighbors.'

Practice Problems

- Given an $m \times n$ grid of non-negative numbers, find the minimum-sum path from top-left to bottom-right, moving only down or right.
- Given a triangle array, find the minimum path sum from top to bottom, moving to one of two adjacent positions in the row below at each step.

LeetCode

- [Minimum Path Sum — leetcode.com/problems/minimum-path-sum](https://leetcode.com/problems/minimum-path-sum)
- [Triangle — leetcode.com/problems/triangle](https://leetcode.com/problems/triangle)

Note: Unique Paths (the canonical introductory Grid DP problem) is referenced conceptually above but not separately assigned, since it has a closed-form combinatorial solution (Phase 9's divide-and-conquer territory) that doesn't require DP — Minimum Path Sum and Triangle are used instead as genuine DP-requiring grid problems.

Pattern 6: Partition DP

Concepts

- Deciding, at each position in a sequence, where to make a 'cut' that splits the remaining problem into an already-solved prefix and a new segment to validate.
- Recognizing the 'can this be built from smaller validated pieces' shape, which appears whenever a string or sequence must be decomposed according to some dictionary or rule.

Practice Problems

- Given a string and a dictionary of words, determine whether the string can be segmented into a sequence of dictionary words.

LeetCode

- [Word Break — leetcode.com/problems/word-break](https://leetcode.com/problems/word-break)

Pattern 7: Tree DP

Concepts

- Computing, for each node in a tree, two related answers (e.g. 'best value including this node' and 'best value excluding this node') in a single bottom-up post-order traversal, then combining at the root.
- Connecting directly to Phase 11's tree-recursion patterns (Diameter, Path Sum): Tree DP is the same bottom-up recursive shape, now carrying two values upward instead of one.

Practice Problems

- Given the root of a binary tree representing houses, where directly connected houses cannot both be robbed, find the maximum amount that can be robbed.

LeetCode

- [House Robber III — leetcode.com/problems/house-robber-iii](https://leetcode.com/problems/house-robber-iii)

Pattern 8: Bitmask DP (Advanced)

Concepts

- Representing a subset of elements (which items have been used, which cities have been visited) as the bits of an integer, enabling $O(1)$ state transitions and set operations that would otherwise require explicit set data structures.
- Recognizing when problem constraints (typically $n \leq \sim 20$) signal that an exponential-looking 2^n state space is intentional and tractable, not a sign the approach is wrong.

Practice Problems

- Given an array of integers and an integer k , determine whether it's possible to divide the array into k non-empty subsets with equal sum, using a bitmask to represent which elements have been assigned.

LeetCode

- [Partition to K Equal Sum Subsets — leetcode.com/problems/partition-to-k-equal-sum-subsets](https://leetcode.com/problems/partition-to-k-equal-sum-subsets)

Phase Assessment

Implementation Tasks

- Convert a naive exponential recursive solution to memoized, then to fully tabulated, and compare measured running times on increasing input sizes.
- Implement a space-optimized rolling-array version of a 2D DP solution and verify it matches the full-table version's output.
- Analyze and explain, using Partition Equal Subset Sum, exactly why the fractional-knapsack greedy strategy from Phase 15 cannot be applied to 0/1 knapsack-shaped problems.

Recommended LeetCode Target

Medium Problems

1D DP

- leetcode.com/problems/house-robber

2D DP

- leetcode.com/problems/longest-common-subsequence

Knapsack Pattern

- leetcode.com/problems/partition-equal-subset-sum

LIS Pattern

- leetcode.com/problems/longest-increasing-subsequence

Grid DP

- leetcode.com/problems/minimum-path-sum
- leetcode.com/problems/triangle

Partition DP

- leetcode.com/problems/word-break

Hard Problems

Tree DP

- leetcode.com/problems/house-robber-iii

Bitmask DP

- leetcode.com/problems/partition-to-k-equal-sum-subsets

On scope and dedup

This phase has 8 distinct problems covering all 8 patterns, with 2 Hard problems instead of the usual 1, since Tree DP and Bitmask DP are both meaningfully harder than the other six patterns and neither has a good Medium-difficulty substitute without losing the pattern's defining complexity. Unique Paths was deliberately not assigned despite being the most commonly cited 'Grid DP' example: it has an $O(1)$ closed-form combinatorial solution, so it doesn't actually require DP to solve optimally, and Minimum Path Sum / Triangle are used instead as problems where DP is genuinely necessary. None of the 8 problems overlaps with any problem used in Phases 0–15.

Coding Challenges (Assessment Day)

- House Robber
- Longest Common Subsequence
- Partition Equal Subset Sum
- Longest Increasing Subsequence
- Word Break
- House Robber III

Expected Outcome

Students should be able to:

- Identify overlapping subproblems and optimal substructure in a new problem, and design appropriate DP state and transitions from scratch.
- Implement DP solutions both top-down (memoized) and bottom-up (tabulated), and choose space-optimized variants when appropriate.
- Recognize and apply all eight DP shapes covered: 1D, 2D, knapsack, LIS, grid, partition, tree, and bitmask.
- Distinguish problems that require DP from superficially similar problems solvable by greedy (Phase 15) or closed-form combinatorics (Phase 9).